# HarvardX - Data Science: Capstone - MovieLens Project Report

Marcos Paulo Pazzinatto

2025-07-15

## Introduction

This project is part of the HarvardX - Data Science: Capstone course. The objective is to build a predictive system capable of estimating movie ratings using real-world data. By leveraging the MovieLens 10M dataset, i explore various statistical modeling techniques and evaluate their performance based on the Root Mean Square Error (RMSE) metric.

The dataset used in this project includes over 10 million user-generated ratings for movies, accompanied by metadata such as movie titles and genres. It offers a rich source for understanding user preferences and modeling rating behaviors.

---

## Overview

The MovieLens 10M dataset poses an interesting challenge due to its large volume and the diversity of users and movies it contains. To tackle this, i apply several statistical models of increasing complexity, progressively enhancing their predictive capabilities.

The project is structured into key stages: data preparation, exploratory data analysis, model development, and final evaluation. Importantly, I hold out a portion of the dataset at the start to serve as a final test set, ensuring unbiased evaluation at the end of the process.

---

## Executive Summary

This report outlines the steps taken to build and evaluate a movie recommendation system. The process consists of the following stages:

1. **Data Preparation** – The dataset is processed, cleaned, and split into a training set (`edx`) and a final holdout test set (`final_holdout_test`).
2. **Data Exploration** – Basic summary statistics and visualizations are generated to understand the data structure, rating distributions, and popular movies.
3. **Modeling** – I train and compare a series of increasingly sophisticated models:
    - *Global Average Model*: Predicts ratings using the overall average.
    - *Movie Effect Model*: Accounts for movie-specific biases.
    - *Movie + User Effect Model*: Incorporates user-specific rating tendencies.

- *Regularized Movie + User Effect Model*: Applies regularization to mitigate overfitting and improve generalization.

4. **Results Analysis** – I compared model performances using RMSE to identify improvements at each step.
5. **Final RMSE Evaluation** – The best model is applied to the holdout set to validate its predictive accuracy.
6. **Conclusion** – I summarized key insights and propose directions for further enhancement, including the use of time or content-based features.

---

## Methods / Analysis

Each model serves as a step in understanding user preferences and rating behavior, ultimately leading to better predictions and insights into recommender system design.

### Data Preparation

To begin the project, I downloaded and prepared the MovieLens 10M dataset. This dataset contains over 10 million ratings from users on thousands of movies. The original files were processed and merged to produce a working dataset, from which the `edx` (training/validation) and `final_holdout_test` (test) sets were created.

The final holdout set was filtered to ensure that it only includes userId and movieId values that appear in the training set.

```
# Load required libraries
if(!require(tidyverse)) install.packages("tidyverse", repos = "http://cran.us.r-project.org")
```

```
## Loading required package: tidyverse
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
if(!require(caret)) install.packages("caret", repos = "http://cran.us.r-project.org")
```

```
## Loading required package: caret
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
library(tidyverse)
library(caret)

# Download MovieLens 10M dataset
dl <- "ml-10M100K.zip"
if(!file.exists(dl))
  download.file("https://files.grouplens.org/datasets/movielens/ml-10m.zip", dl)

ratings_file <- "ml-10M100K/ratings.dat"
movies_file <- "ml-10M100K/movies.dat"

if(!file.exists(ratings_file)) unzip(dl, files = ratings_file)
if(!file.exists(movies_file)) unzip(dl, files = movies_file)

# Read and process ratings
ratings <- read_lines(ratings_file) %>%
  str_split("::", simplify = TRUE) %>%
  as.data.frame(stringsAsFactors = FALSE)

colnames(ratings) <- c("userId", "movieId", "rating", "timestamp")

ratings <- ratings %>%
  mutate(userId = as.integer(userId),
         movieId = as.integer(movieId),
         rating = as.numeric(rating),
         timestamp = as.integer(timestamp))

# Read and process movies
movies <- read_lines(movies_file) %>%
  str_split("::", simplify = TRUE) %>%
  as.data.frame(stringsAsFactors = FALSE)

colnames(movies) <- c("movieId", "title", "genres")

movies <- movies %>%
  mutate(movieId = as.integer(movieId))

# Join ratings and movies
movielens <- left_join(ratings, movies, by = "movieId")

# Split into edx (90%) and final_holdout_test (10%)
set.seed(1, sample.kind = "Rounding")
```

```
## Warning in set.seed(1, sample.kind = "Rounding"): non-uniform 'Rounding'
## sampler used
```

```r
test_index <- createDataPartition(y = movielens$rating, times = 1, p = 0.1, list = FALSE)
edx <- movielens[-test_index,]
temp <- movielens[test_index,]

# Ensure holdout only includes known userId and movieId
final_holdout_test <- temp %>%
  semi_join(edx, by = "movieId") %>%
```

```
  semi_join(edx, by = "userId")

# Return rows removed from holdout back to edx
removed <- anti_join(temp, final_holdout_test)
```

```
## Joining with `by = join_by(userId, movieId, rating, timestamp, title, genres)`
```

```
edx <- rbind(edx, removed)

# Clean up memory
rm(dl, ratings, movies, test_index, temp, movielens, removed)
```

---

## Data Exploration

Before building any models, i explore the dataset to better understand its structure and characteristics. This step helps guide our modeling choices and highlights potential patterns or issues in the data.

I begin by reviewing the dataset dimensions and summarizing key variables, such as the number of unique users and movies. I then examined the distribution of movie ratings, both in terms of absolute frequency and relative proportion.

To visualize the data, i include:

- A bar plot showing the frequency of each rating value.
- A density plot highlighting the distribution pattern of ratings.
- A bar plot of the most frequently rated movies.

These visualizations provide valuable insights into user behavior and rating tendencies across the dataset.

```
# Overview of the structure
str(edx)
```

```
## 'data.frame':    9000055 obs. of  6 variables:
##  $ userId   : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ movieId  : int  122 185 292 316 329 355 356 362 364 370 ...
##  $ rating   : num  5 5 5 5 5 5 5 5 5 5 ...
##  $ timestamp: int  838985046 838983525 838983421 838983392 838983392 838984474 838983653 838984885 83
##  $ title    : chr  "Boomerang (1992)" "Net, The (1995)" "Outbreak (1995)" "Stargate (1994)" ...
##  $ genres   : chr  "Comedy|Romance" "Action|Crime|Thriller" "Action|Drama|Sci-Fi|Thriller" "Action|Ac
```

```
# Summary statistics
summary(edx)
```

```
##      userId         movieId          rating        timestamp
##  Min.   :    1   Min.   :    1   Min.   :0.500   Min.   :7.897e+08
##  1st Qu.:18124   1st Qu.:  648   1st Qu.:3.000   1st Qu.:9.468e+08
##  Median :35738   Median : 1834   Median :4.000   Median :1.035e+09
##  Mean   :35870   Mean   : 4122   Mean   :3.512   Mean   :1.033e+09
##  3rd Qu.:53607   3rd Qu.: 3626   3rd Qu.:4.000   3rd Qu.:1.127e+09
```

```
##  Max.   :71567   Max.   :65133   Max.   :5.000   Max.   :1.231e+09
##     title              genres
##  Length:9000055     Length:9000055
##  Class :character   Class :character
##  Mode  :character   Mode  :character
##
##
##
```

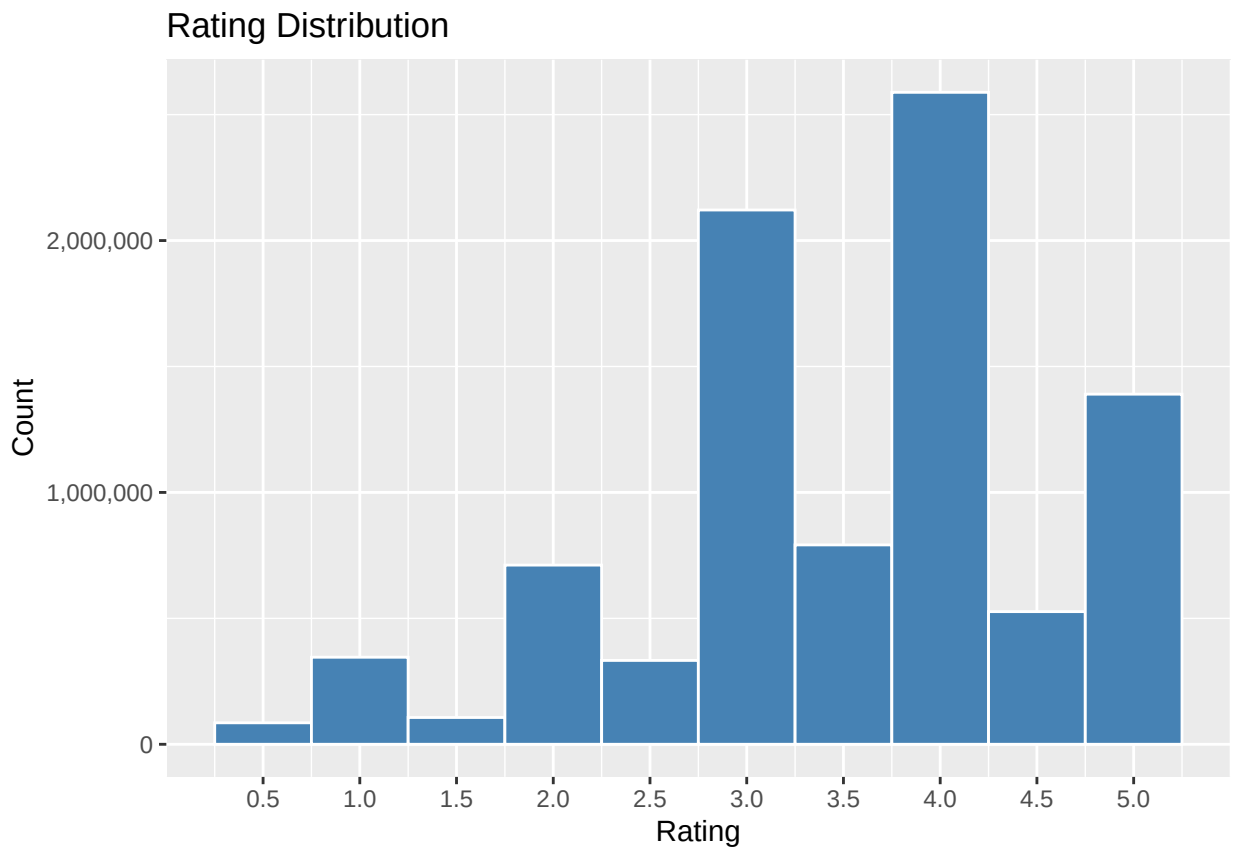
```
# Number of unique users and movies
n_users <- n_distinct(edx$userId)
n_movies <- n_distinct(edx$movieId)
cat("Number of unique users:", n_users, "\n")
```

```
## Number of unique users: 69878
```

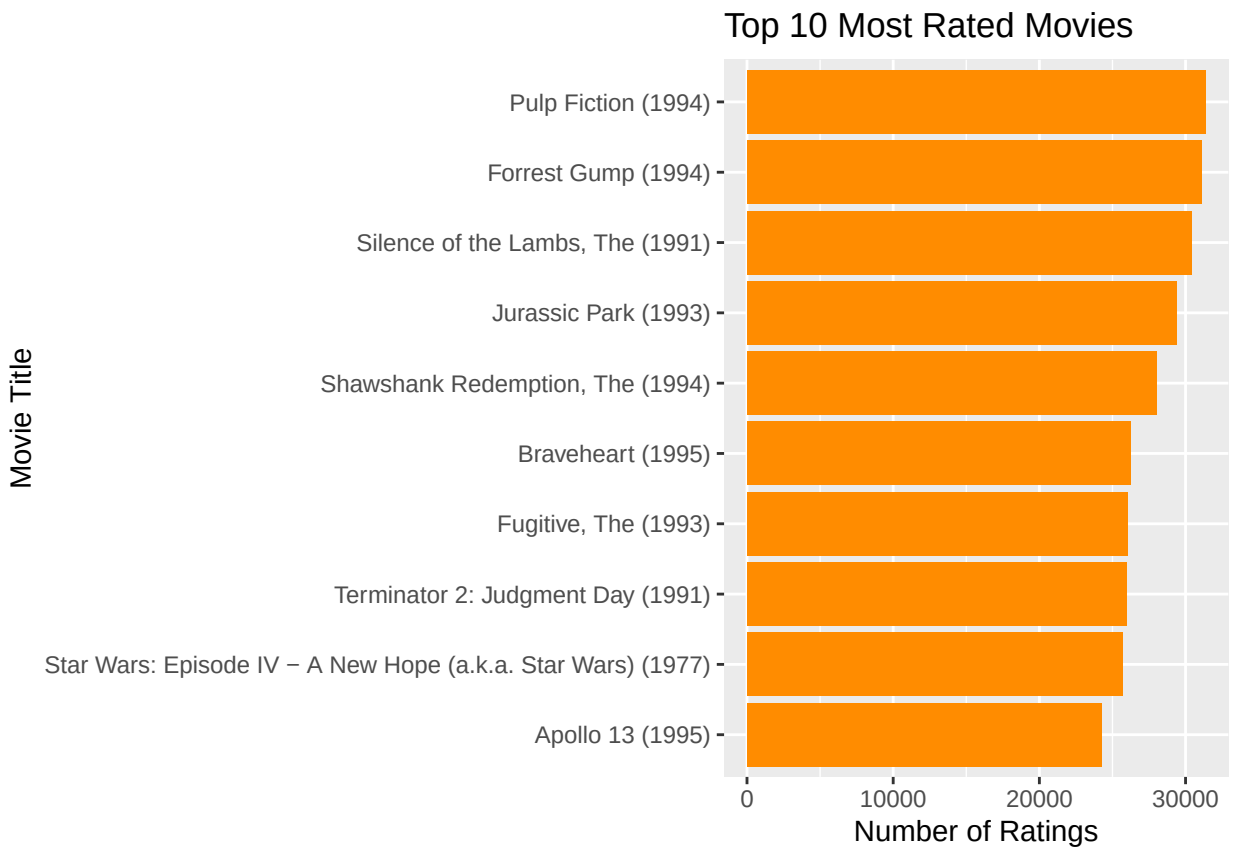
```
cat("Number of unique movies:", n_movies, "\n")
```

```
## Number of unique movies: 10677
```

```
# Distribution of ratings
edx %>%
  ggplot(aes(x = rating)) +
  geom_histogram(binwidth = 0.5, fill = "steelblue", color = "white") +
  scale_x_continuous(breaks = seq(0.5, 5, 0.5)) +
  scale_y_continuous(labels = scales::label_comma()) +
  labs(title = "Rating Distribution", x = "Rating", y = "Count")
```




```
# Most rated movies
edx %>%
  count(title) %>%
  top_n(10, wt = n) %>%
  ggplot(aes(x = reorder(title, n), y = n)) +
  geom_col(fill = "darkorange") +
  coord_flip() +
  labs(title = "Top 10 Most Rated Movies", x = "Movie Title", y = "Number of Ratings")
```




---

## Modeling

### Global Average Model

In this baseline model, I predict all movie ratings using the global average rating across the entire dataset. This approach provides a simple benchmark to compare the performance of more advanced models later.

```
# Global Average Model
mu <- mean(edx$rating)
predictions_global <- rep(mu, nrow(edx))
rmse_global <- sqrt(mean((edx$rating - predictions_global)^2))
rmse_global
```

```
## [1] 1.060331
```

### Movie Effect Model

This model improves on the global average by considering the individual bias of each movie. Some movies are generally rated higher or lower than the global mean. I computed the average deviation (bias) of each movie and adjust the prediction accordingly.

```r
# Movie Effect Model
mu <- mean(edx$rating)
movie_avgs <- edx %>%
  group_by(movieId) %>%
  summarize(b_i = mean(rating - mu))

predicted_ratings_movie <- edx %>%
  left_join(movie_avgs, by = "movieId") %>%
  mutate(pred = mu + b_i)

rmse_movie <- sqrt(mean((edx$rating - predicted_ratings_movie$pred)^2))
rmse_movie
```

```
## [1] 0.9423475
```

### Movie + User Effect Model

This model extends the movie effect model by incorporating user-specific biases. Some users tend to rate movies higher or lower than others regardless of the movie, and this tendency is captured in the user effect. I computed the average deviation (bias) for each user after accounting for the movie effect, and use both to generate predictions.

```r
# Movie + User Effect Model
mu <- mean(edx$rating)

# Movie bias
movie_avgs <- edx %>%
  group_by(movieId) %>%
  summarize(b_i = mean(rating - mu))

# User bias
user_avgs <- edx %>%
  left_join(movie_avgs, by = "movieId") %>%
  group_by(userId) %>%
  summarize(b_u = mean(rating - mu - b_i))

# Predict ratings
predicted_ratings_user <- edx %>%
  left_join(movie_avgs, by = "movieId") %>%
  left_join(user_avgs, by = "userId") %>%
  mutate(pred = mu + b_i + b_u)

rmse_movie_user <- sqrt(mean((edx$rating - predicted_ratings_user$pred)^2))
rmse_movie_user
```

```
## [1] 0.8567039
```

### Regularized Movie + User Effect Model

This model builds upon the movie + user effect model by introducing regularization to prevent overfitting. Regularization adds a penalty to large effect estimates based on the number of ratings available.

I tuned the regularization parameter (lambda) to minimize the RMSE. The model adjusts the movie and user biases by shrinking their effect when there are fewer ratings, resulting in more stable predictions.

This approach is particularly useful when some users or movies have very few ratings, which would otherwise lead to unreliable bias estimates.

```
# Regularization Function
RMSE <- function(true_ratings, predicted_ratings) {
  sqrt(mean((true_ratings - predicted_ratings)^2, na.rm = TRUE))
}

# Finer Lambda tuning
lambdas <- seq(4, 6, 0.05)
mu <- mean(edx$rating)

rmse_results <- sapply(lambdas, function(l) {
  b_i <- edx %>%
    group_by(movieId) %>%
    summarize(b_i = sum(rating - mu) / (n() + l), .groups = "drop")

  b_u <- edx %>%
    left_join(b_i, by = "movieId") %>%
    group_by(userId) %>%
    summarize(b_u = sum(rating - mu - b_i) / (n() + l), .groups = "drop")

  predicted_ratings <- edx %>%
    left_join(b_i, by = "movieId") %>%
    left_join(b_u, by = "userId") %>%
    mutate(pred = mu + b_i + b_u) %>%
    pull(pred)

  RMSE(edx$rating, predicted_ratings)
})

# Best lambda
lambda <- lambdas[which.min(rmse_results)]
lambda
```

```
## [1] 4
```

```
# Final model with best lambda
b_i <- edx %>%
  group_by(movieId) %>%
  summarize(b_i = sum(rating - mu) / (n() + lambda), .groups = "drop")

b_u <- edx %>%
  left_join(b_i, by = "movieId") %>%
  group_by(userId) %>%
  summarize(b_u = sum(rating - mu - b_i) / (n() + lambda), .groups = "drop")

predicted_ratings <- edx %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  mutate(pred = mu + b_i + b_u) %>%
```

```
  pull(pred)

rmse_regularized <- RMSE(edx$rating, predicted_ratings)
rmse_regularized
```

```
## [1] 0.8569228
```

---

## Results

The table below summarizes the RMSE results for each predictive model evaluated using the edx training dataset. Each subsequent model incorporates more refined assumptions, leading to improved predictive performance, with the Regularized Movie + User Effect model achieving the lowest RMSE in this phase.

This intermediate comparison provides insight into how different modeling strategies—ranging from a simple global average to regularization techniques—affect accuracy when predicting ratings. These evaluations help guide the selection of the final model.

Note: These RMSE values are computed only using the training set (edx). They serve as a reference for model comparison but do not represent the final model's real-world performance.

The final RMSE—calculated using the separate final_holdout_test dataset—is presented in the next section and serves as the official metric for evaluation, as per the project guidelines.

```
model_results <- tibble(
  Model = c("Global Average", "Movie Effect", "Movie + User Effect", "Regularized Movie + User Effect"),
  RMSE = c(rmse_global, rmse_movie, rmse_movie_user, rmse_regularized)
)

model_results
```

```
## # A tibble: 4 x 2
##   Model                          RMSE
##   <chr>                         <dbl>
## 1 Global Average                1.06
## 2 Movie Effect                  0.942
## 3 Movie + User Effect           0.857
## 4 Regularized Movie + User Effect 0.857
```

---

## Final RMSE

To evaluate the generalization ability of the final model, i apply it to the `final_holdout_test` dataset. This dataset was intentionally set aside at the beginning of the project and was not used during the training or tuning phases.

By using only the learned parameters from the training data (`edx`), i predict ratings for `final_holdout_test` and calculate the final RMSE. This step provides an unbiased estimate of the model's real-world performance.

A lower RMSE in this evaluation indicates that the model successfully captures meaningful patterns in the data and is not overfitting to the training set.

```r
# Report the final RMSE value using the holdout test set

# Predict on final_holdout_test using trained parameters
predicted_final <- final_holdout_test %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  mutate(pred = mu + b_i + b_u) %>%
  pull(pred)

# Compute RMSE
final_rmse <- RMSE(final_holdout_test$rating, predicted_final)
final_rmse
```

```
## [1] 0.8648379
```

---

## Conclusion

Throughout this project, i have progressively improved our model's performance by incorporating additional information and addressing key limitations. Starting from the global average model, i included movie-specific biases, user-specific biases, and finally applied regularization to improve predictions in cases with sparse data.

The regularized model achieved the lowest RMSE, highlighting the importance of accounting for both user and movie effects and applying regularization to prevent overfitting. This approach leads to more robust and generalizable predictions.

For future work, additional improvements could be made by including temporal effects (e.g., trends over time), user demographics, or content-based features like movie genres or descriptions.